

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 3: Estrutura MVC aplicada ao desenvolvimento web

Do problema à arquitetura: por que organizar o código em camadas

Aula prática: projeto do zero, do formulário ao servidor

Objetivos da aula

1

Entender o problema que o MVC resolve

Código misturado (lógica + HTML + dados) em um único lugar.

2

Explicar Model, View e Controller

O papel de cada camada e como conversam entre si.

3

Comparar MVC com outros padrões

MVP, MVVM e a diferença entre MVC e microserviços.

4

Construir um projeto do zero

Reaproveitando o formulário HTML da Aula 1, agora com Servlet processando os dados.

Agenda da aula

1 O problema: um Servlet fazendo tudo

2 O que é MVC — Model, View, Controller

3 O fluxo de uma requisição no MVC

4 Por que usar MVC

5 Comparativo: MVC vs MVP vs MVVM

6 MVC não é microserviços

7 Estrutura de pastas de um projeto organizado

8 Prática: construindo o projeto do zero

O problema: um Servlet fazendo tudo

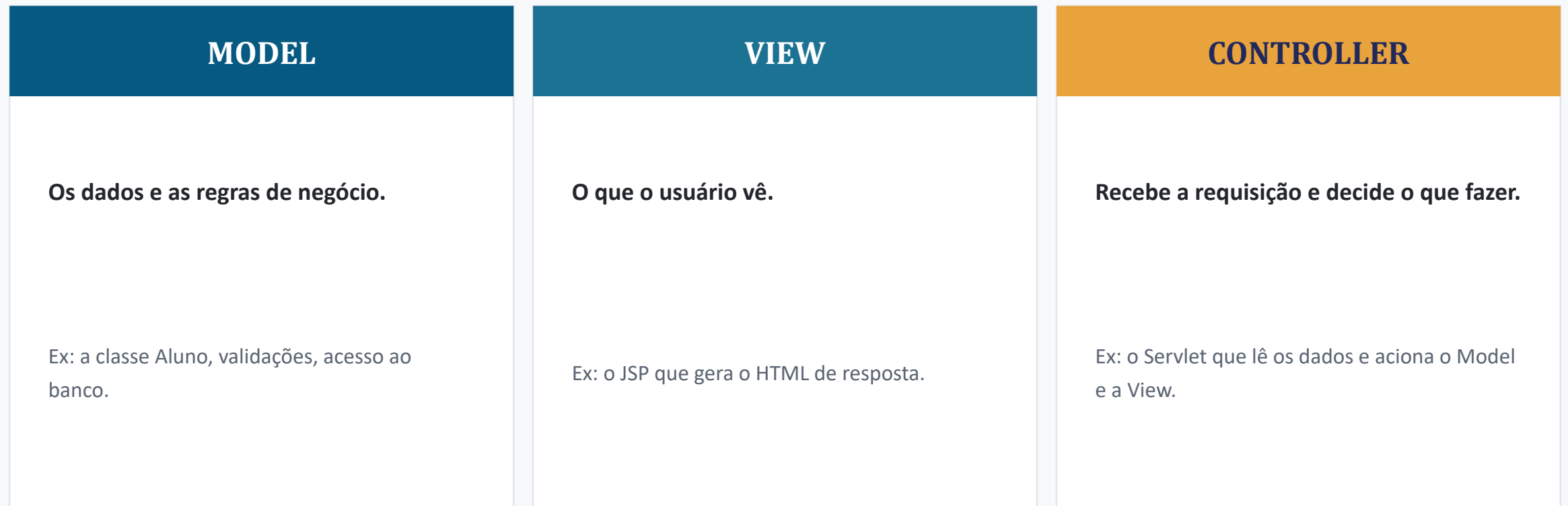
Sem organização, um único Servlet acumula responsabilidades demais:

```
protected void doPost(HttpServletRequest req, HttpServletResponse res) {  
    String nome = req.getParameter("nome");           // 1. lê dados  
    // valida o nome...                               // 2. valida  
    Connection conn = DriverManager.getConnection(...); // 3. acessa banco  
    conn.prepareStatement("INSERT INTO aluno...");     // 4. grava dados  
    res.setContentType("text/html");  
    res.getWriter().println("<html><body><h1>Ok, "      // 5. monta HTML  
        + nome + "!</h1></body></html>");  
}
```

Consequências práticas: difícil de testar, difícil de reaproveitar, difícil de dar manutenção — e impossível dividir o trabalho entre pessoas diferentes da equipe sem todo mundo mexer no mesmo arquivo.

O que é MVC

MVC (Model-View-Controller) é um padrão que separa uma aplicação em três responsabilidades independentes:



O FLUXO

O caminho de uma requisição no MVC



O Controller nunca gera HTML — ele repassa o processamento adiante com:

```
request.getRequestDispatcher("resultado.jsp").forward(request, response);
```

O forward mantém a mesma requisição (o Model fica disponível na View via request.setAttribute).

Por que usar MVC

Separação de responsabilidades

Cada camada tem um único motivo para mudar.

Manutenibilidade

Alterar a tela não arrisca quebrar a regra de negócio, e vice-versa.

Testabilidade

O Model pode ser testado sozinho, sem precisar de navegador.

Trabalho em equipe paralelo

Uma pessoa mexe na View, outra no Controller, outra no Model — sem conflito constante.

Reuso

O mesmo Model pode alimentar mais de uma View (ex: HTML e JSON).

MVC vs. MVP vs. MVVM

	MVC	MVP	MVVM
Quem fala com a View	Controller (indireto)	Presenter (direto, via interface)	ViewModel (via data binding)
Onde fica a lógica de apresentação	Espalhada entre Controller e View	Concentrada no Presenter	Concentrada na ViewModel
Onde é mais comum	Apps web tradicionais (JSP, Spring MVC)	Apps desktop/mobile mais antigas	Apps com binding automático (Angular, WPF)

Para a disciplina, usaremos o MVC clássico com Servlet (Controller) e JSP (View).

MVC não é a mesma coisa que microserviços

MVC

É um padrão de organização do código DENTRO de uma aplicação — separa responsabilidades de apresentação, dados e controle.

Continua sendo UM único sistema, implantado como UMA aplicação.

MICROSSERVIÇOS

É um estilo de arquitetura de SISTEMA — divide a aplicação em vários serviços independentes, cada um implantado separadamente.

Cada microserviço, por dentro, ainda pode (e costuma) usar MVC.

Resumo: um resolve "como organizar o código"; o outro resolve "como dividir o sistema entre times e servidores". São decisões em níveis diferentes, e não concorrem entre si.

Estrutura de pastas de um projeto MVC

```
src/main/java/  
  br/edu/faculdade/  
    model/  
      Aluno.java  
    controller/  
      CadastroAlunoServlet.java  
    dao/  
      (próximas aulas: JDBC)  
  
src/main/webapp/  
  cadastro.jsp  
  resultado.jsp  
  WEB-INF/  
    web.xml
```

CADA PASTA, UMA RESPONSABILIDADE

- model — classes de dados (o "substantivo" do sistema)
- controller — Servlets que recebem requisições
- dao — acesso a banco de dados (próximas aulas)
- webapp — páginas JSP (a View)

Nomear pastas pelo papel que exercem (não pelo tipo de tela) facilita muito a navegação em projetos maiores.

Vamos construir do zero

Retomando o formulário HTML da Aula 1 (cadastro de aluno), agora vamos processá-lo de verdade, com um Servlet estruturado em MVC.

1. Model

Uma classe Aluno com os campos nome, email e data de nascimento.

2. Controller

Um Servlet que recebe o POST, cria o Aluno e encaminha para a View.

3. View

Uma página JSP de resultado, exibindo os dados recebidos.

Projeto: `aula3-mvc-cadastro` (Maven webapp, mesmo padrão de groupId `br.edu.faculdade` das aulas anteriores)

Reaproveite o `cadastro.jsp` que vocês fizeram no exercício da Aula 1 — apenas confirme que o action e os name dos campos batem com o Servlet.

1 Model: a classe Aluno

src/main/java/br/edu/faculdade/model/Aluno.java

```
import java.time.LocalDate;

public class Aluno {
    private String nome;
    private String email;
    private LocalDate nascimento;

    public Aluno(String nome, String email, LocalDate nascimento) {
        this.nome = nome;
        this.email = email;
        this.nascimento = nascimento;
    }

    public String getNome() { return nome; }
    public String getEmail() { return email; }
    public LocalDate getNascimento() { return nascimento; }
}
```

2

Controller: o Servlet

src/main/java/br/edu/faculdade/controller/CadastroAlunoServlet.java

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import br.edu.faculdade.model.Aluno;
import java.time.LocalDate;

@WebServlet("/cadastro-aluno")
public class CadastroAlunoServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws Exception {
        String nome = request.getParameter("nome");
        String email = request.getParameter("email");
        LocalDate nascimento =
            LocalDate.parse(request.getParameter("nascimento"));

        Aluno aluno = new Aluno(nome, email, nascimento);
        request.setAttribute("aluno", aluno);

        request.getRequestDispatcher("resultado.jsp")
            .forward(request, response);
    }
}
```

3

View: resultado.jsp

src/main/webapp/resultado.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<body>
  <h1>Aluno cadastrado com sucesso!</h1>
  <p>Nome: ${aluno.nome}</p>
  <p>Email: ${aluno.email}</p>
  <p>Nascimento: ${aluno.nascimento}</p>
</body>
</html>
```

`${aluno.nome}` é EL (Expression Language) — busca automaticamente o atributo "aluno" que o Controller colocou na requisição e chama `getNome()` por trás dos panos. É a forma recomendada hoje, no lugar de scriptlets (`<%= %>`).

RESUMO

O que construímos hoje

- 1 cadastro.jsp (View) envia o formulário via POST
- 2 CadastroAlunoServlet (Controller) recebe, cria o Aluno (Model)
- 3 O Controller encaminha para resultado.jsp (View) com forward
- 4 resultado.jsp exibe os dados usando Expression Language

Próxima aula: criação de endpoints REST — expondo dados também em JSON, não só em HTML.